MANUAL CANÔNICO DE ENGENHARIA DE AGENTES

Guia Definitivo de Agent Skills & Orquestração Autônoma

A síntese definitiva integrando o framework de 3 camadas,
fan-out concorrente no /ship, padrão Prove-It e matriz de restrições.



Base de Conhecimento: addyosmani/agent-skills · Claude Code · Hermes Agent · Antigravity

Arquitetura & Engenharia: Augusto Henrique (AH3 BigBrain)

FASE 1: FLUXO OPERACIONAL PADRÃO (V1)

Os 4 Gates Inegociáveis do Ciclo de Vida

Nenhum código é escrito sem especificação prévia. O agente avança estritamente através de gates formais.

1. SPEC

Definição de Contratos

Skill: spec-driven-development
+ api-and-interface-design

- Formalizar .spec.md
- Definir inputs, outputs e payloads
- Mapear casos de erro de borda

Nenhuma linha de código é criada antes da aprovação do spec

2. PLAN

Fatiamento Atômico

Skill: planning-and-task-breakdown
+ constraint-driven-dev

- Decompor em passos curtos
- Escopo limitado a 1 arquivo/etapa
- Definir ordem de dependência

Evita regressões e perda de contexto durante a implementaçã

3. BUILD

Implementação com TDD

Skill: incremental-implementation
+ test-driven-development

- Codificação incremental
- Testes a cada novo bloco
- Validação com terminal real

Compilação verde e linter limpo são obrigatórios a cada etapa.

4. VERIFY

Revisão & Qualidade

Skill: code-review-and-quality
+ code-simplification

- Análise em 5 eixos de qualidade
- Remoção de código redundante
- Verificação de regressões

Código só avança para merge após validação de todos os critérios.

FASE 2: PREPARAÇÃO DE RELEASE (V2)

Comando /ship: Multi-Agent Fan-Out Concorrente

Divisão paralela para 3 especialistas com contexto isolado, culminando em uma decisão binária de Go / No-Go.

code-reviewer
Senior Staff Engineer

- Análise de complexidade ciclomática
- Identificação de código duplicado
- Manutenibilidade e contratos de API
- Conformidade com design patterns

security-auditor
Security Engineer

- Varredura de OWASP Top 10 e CWEs
- Prevenção contra injeção e XSS
- Auditoria de credenciais e segredos
- Sanitização estrita de entradas

test-engineer
QA Engineer

- Aplicação do padrão Prove-It
- Cobertura de caminhos críticos
- Validação de casos de borda reais
- Verificação de regressões ocultas

FASE DE SÍNTESE (MERGE PHASE) & CRITÉRIOS DE VETO

VETO AUTOMÁTICO (NO-GO): Qualquer falha de segurança 'High/Critical', teste quebrado ou objeção arquitetural bloqueia o PR imediatamente.

APROVAÇÃO (GO): 3 pareceres favoráveis emitidos de forma independente + plano de rollback documentado para a release.

FASE 3 & 4: ROBUSTEZ EXTREMA (V3 & V4)

Padrão 'Prove-It' para Bugs e Matriz de Restrições

Verificação empírica obrigatória para código crítico e travas rígidas de performance e segurança.

PADRÃO 'PROVE-IT' (BUGS & CRITICAL)

Ciclo RED-GREEN-REFACTOR

1. PROVA DE FALHA (RED):
Escrever teste unitário reproduzindo o bug. O teste DEVE FALHAR no terminal antes de qualquer patch.

2. CORREÇÃO MÍNIMA (GREEN):
Alterar estritamente as linhas necessárias para fazer o teste passar (exit code 0).

3. SIMPLIFICAÇÃO (REFACTOR):
Executar 'code-simplification' para eliminar código redundante sem introduzir regressões.

Regra: Zero consertos 'no escuro'. Prova empírica via terminal.

MATRIZ DE RESTRIÇÕES (V4)

Floor-Guard & Não-Funcionais

Performance Web:
INP < 200ms · LCP < 2.5s · CLS < 0.1
Auditado via 'web-performance-auditor' e DevTools.

Bundle & Payload:
Limite máximo de acréscimo de +15KB gzip.
Tree-shaking e code-splitting obrigatórios.

Segurança de Dependências:
Zero vulnerabilidades conhecidas (npm audit 0).
Auditoria contínua com 'security-and-hardening'.

Backend & Latência:
TTFB < 150ms · Zero Memory Leaks em loop.
Monitoramento via 'observability-and-instrumentation'.

FASE 5: MANUAL DE INTEGRAÇÃO (V5)

Intent Engine: Mapeamento Automático de Habilidades

O agente identifica o gatilho da mensagem do usuário e seleciona a rota metodológica ótima.

Criar Nova Funcionalidade

Skill Primária: spec-driven-development
Suporte: planning-and-task-breakdown, api-interface-design

Output Obrigatório: SPEC.md aprovado e checklist atômico de tarefas antes de codificar.

Corrigir Bug ou Falha

Skill Primária: debugging-and-error-recovery
Suporte: test-driven-development (Padrão Prove-It)

Output Obrigatório: Teste reproduzindo o erro no terminal antes da correção mínima.

Refatorar ou Limpar Código

Skill Primária: code-simplification
Suporte: test-driven-development, codebase-inspection

Output Obrigatório: Redução de linhas e complexidade com 100% de aprovação na suite de testes.

Auditar Performance ou UI

Skill Primária: performance-optimization
Suporte: browser-testing-with-devtools, frontend-ui-engineering

Output Obrigatório: Relatório com Core Web Vitals (INP, LCP, CLS) e visual traces de render.

Princípio Operacional: Se uma skill especializada existe para a tarefa, ela DEVE ser invocada.

CONCLUSÃO & AUDITORIA

Checklist de Conformidade Pré-Merge

Critérios absolutos para entrega de software com garantia de qualidade e estabilidade operacional.

CHECKLIST DE SAÍDA DO ENGENHEIRO SÊNIOR

- [ ] **1. Contrato & Especificação Formal**
  A funcionalidade foi especificada em .spec.md antes da criação do código?
- [ ] **2. Prova de Falha Anterior (Prove-It)**
  O teste unitário reproduziu a falha no terminal real antes da correção?
- [ ] **3. Cobertura & Suíte Verde**
  Todos os testes unitários e de integração passaram com exit code 0?
- [ ] **4. Fan-Out Aprovado no /ship**
  As 3 personas especializadas (Reviewer, Security, QA) emitiram parecer favorável?
- [ ] **5. Matriz de Restrições Atendida**
  Os limites de Core Web Vitals, memória, latência e bundle size foram mantidos?
- [ ] **6. Código Enxuto & Convencional**
  A skill code-simplification foi executada e os commits seguem Conventional Commits?